



Road Reorganization (roads)

Het Romeinse leger heeft recent nieuwe rijtuigen verkregen die significant meer mensen kunnen vervoeren. Helaas betekent dit ook dat de rijtuigen te groot zijn en vaker vastzitten als er tegenliggers zijn. Om dit te voorkomen besluit de keizer om elke straat in het keizerrijk eenrichtingsverkeer te maken.

Maar de keizer gaat graag vaak naar zijn geboortedorp, en wil er voor zorgen dat hij altijd snel kan terugkeren naar Rome als er iets belangrijks gebeurt. Daarom wil hij zorgen dat de kortste route van zijn geboortedorp naar Rome niet langer wordt dan het nu is. Bovendien wil hij ook zeker zijn dat hij kan terugkeren naar zijn geboortedorp, maar dit hoeft niet zo snel mogelijk te zijn.

Het Romeinse Rijk bevat N steden en er zijn M tweerichtingsstraten. Het geboortedorp van de keizer is stad 0 en Rome is stad $N - 1$. Formeel wil hij een richting kiezen voor elke straat. Hij wil zeker zijn dat het kortste pad van stad 0 naar stad $N - 1$ niet langer is dan het was voordat de straten eenrichtingsverkeer werden. Verder wil hij ook zeker zijn dat er nog steeds een pad van stad $N - 1$ terug naar stad 0 is. Jouw taak is om de straten zodanig te oriënteren, of de keizer te laten weten dat dit onmogelijk is.

⇒ Het is gegarandeerd dat elke stad in het Romeinse Rijk bereikbaar is vanaf elke andere stad. Merk op dat dit niet het geval hoeft te zijn nadat je de straten georiënteerd hebt.

Merk op dat de straten een oriëntatie geven zodat je van 0 naar $N - 1$ en terug kan gaan, zonder daarbij de lengte van een van de trips te minimaliseren, je nog steeds punten geeft.

Implementatie

Je moet één .cpp bronbestand inleveren.



In de bijlagen van deze taak vind je een sjabloon `roads.cpp` met een voorbeeld implementatie.

Je moet de volgende functie implementeren:

C++

```
vector<pair<int, int>> orient_roads(int N, int M, vector<int> A,
vector<int> B, vector<int> W)
```

- Integer N staat voor het aantal steden.
- Integer M staat voor het aantal straten.
- De arrays A , B en W beschrijven de straten van het Romeinse Rijk: Er is een straat tussen stad $A[i]$ en stad $B[i]$ met lengte $W[i]$.
- Als er een manier is om de straten te oriënteren, moet de functie een vector van pairs teruggeven van lengte M . Hier staat de i -de pair $\{F, T\}$ voor een straat die georiënteerd wordt van F naar T .
- Als er geen manier is om de straten te oriënteren, moet de functie een lege vector teruggeven.

Het is gegarandeerd dat er hoogstens één straat is tussen twee steden, en dat voor alle i geldt dat $A[i] \neq B[i]$. Als de functie geen vector van lengte 0 of M teruggeeft telt dit als een fout antwoord.

Alle straten die in de invoer vermeldt worden moeten exact één keer aanwezig zijn in de uitvoer, en geen andere straten mogen aanwezig zijn. De volgorde van de straten in de uitvoer vector maakt niet uit.

Voorbeeld Grader

In de bijlagen van deze taak vind je een versimpelde versie van de grader die gebruikt wordt tijdens de evaluatie, die je kunt gebruiken om je oplossingen lokaal te testen. De voorbeeld grader leest data van `stdin`, roept de functie `orient_roads` aan en schrijft uitvoer naar `stdout` in het volgende formaat.

De invoer bestaat uit $M + 1$ regels, waaronder:

- Regel 1: de integers N en M .
- regel $2 + i$ ($0 \leq i < M$): de integers A_i , B_i en W_i .

De uitvoer bestaat uit ofwel 1, ofwel $M + 1$ regels, waaronder:

- Regel 1: **Yes** of **No**, wat aangeeft of het mogelijk is de wegen te oriënteren.
- Als het antwoord **Yes** is: Regel $2 + i$ ($0 \leq i < M$): F_i en T_i , die aangeven dat de straat tussen deze twee steden georiënteerd wordt van F_i naar T_i .

In de uitvoer worden de oriënteringen afgedrukt in de volgorde waarin de werden teruggegeven door de functie.

Randvoorwaarden

- $3 \leq N \leq 100\,000$.
- $N - 1 \leq M \leq 100\,000$.
- $0 \leq A_i, B_i \leq N - 1$ en $A_i \neq B_i$.
- $1 \leq W_i \leq 1\,000\,000\,000$.
- Het is gegarandeerd dat elke stad in het Romeinse Rijk bereikbaar is vanaf elke andere stad.

Scoring

Je programma zal getest worden met meerdere testgevallen gegroepeerd in subtaken. Om een volledige score voor een subtaak te krijgen moet je programma alle testgevallen daarvan correct oplossen. Als de oplossing geldig maar niet optimaal is, krijg je de helft van de punten voor de subtaak.

- **Subtask 0 [0 punten]**: Voorbeeld testgevallen.
- **Subtask 1 [6 punten]**: $\max(N, M) \leq 20$.
- **Subtask 2 [11 punten]**: $M \leq N$.
- **Subtask 3 [17 punten]**: Er wordt gegarandeerd dat er een rechtstreekse straat tussen stad 0 en stad $N - 1$ is.
- **Subtask 4 [18 punten]**: $\max(N, M) \leq 2000$.
- **Subtask 5 [17 punten]**: $W_i = 1$.
- **Subtask 6 [31 punten]**: Geen extra randvoorwaarden.

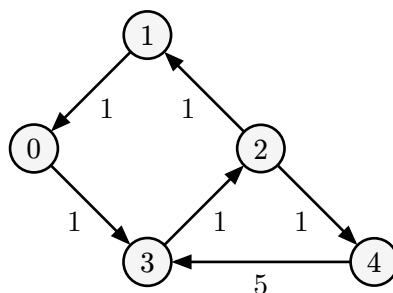


Je kan de helft van de punten voor een testgeval krijgen als het kortste pad tussen stad 0 en $N - 1$ nadat de straten georiënteerd werden niet dezelfde lengte is als het kortste pad van stad 0 naar $N - 1$ voordien, zelfs als er geen manier was om de straten te oriënteren die de keizer volledig gelukkig maakt.

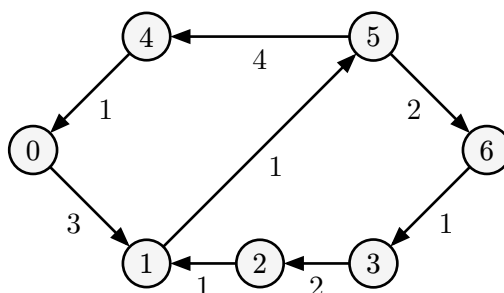
Voorbeelden

stdin	stdout
5 6 0 1 1 1 2 1 0 3 1 3 2 1 2 4 1 3 4 5	Yes 1 0 2 1 0 3 3 2 2 4 4 3
7 8 0 1 3 1 2 1 2 3 2 3 6 1 0 4 1 4 5 4 5 6 2 1 5 1	Yes 0 1 2 1 3 2 6 3 4 0 5 4 5 6 1 5
8 9 0 1 3 1 2 4 2 4 2 0 3 6 3 4 1 4 5 3 5 6 1 6 7 2 5 7 3	No

Uitleg



In het eerste voorbeeld kunnen we van stad 0 naar 4 via $0 \rightarrow 3 \rightarrow 2 \rightarrow 4$. Dit is een pad met lengte 3, wat eveneens de kortst mogelijk lengte was voor de straten georiënteerd werden. We kunnen nog steeds teruggaan via $4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0$. Het is niet van belang hoe lang dit pad is.



In het tweede voorbeeld kunnen we van 0 naar 6 met de route $0 \rightarrow 1 \rightarrow 5 \rightarrow 6$ en terug langs $6 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 4 \rightarrow 0$.

In het derde voorbeeld kan men aantonen dat geen oriëntering van de straten mogelijk is die de keizer gelukkig zou maken.